SESSION 6

Context Engineering II — Tools & Memory

Everything the model can act on or recall is context you design. Plus: the capstone.

Jeudi 24 septembre · 11h00–13h30 · 2h30

Prompt & Context Engineering — Stefan Duprey

TODAY

Session 6 — plan & goals

You will be able to

- Open and review pull requests
- Treat tools & memory as context
- Write tool descriptions as prompts
- Set up a team repo & start the capstone

RUN OF SHOW

0:00	Git — pull requests & code review
0:30	Tools & memory as context + lab
1:50	Capstone brief + team repos

GIT

Pull requests & code review

1

Push a branch

Send your feature branch to the remote / your fork.

2

Open a PR

Describe what changed and why; link the task.

3

Review

Teammates comment, request changes, approve.

4

Merge

Squash or merge once approved — main stays healthy.

Collaborating on the capstone

- Form teams around one shared repository
- Each member works on their own branch
- Integrate through pull requests — never email files
- Review each other's prompts and code

SET UP NOW

Create your team repo today. You'll deliver the whole capstone through it.

REFRAME

Tools & memory are context too

The same lesson as retrieval, extended: what the model can do and what it remembers are also context you design.

- **A tool is context that acts**

  Giving the model a tool doesn't just add a capability — it adds text (the tool's description) that shapes what the model decides to do.

- **Memory is context you persist**

  Anything you remember and re-inject next turn is a deliberate choice about what fills the window later.

- **Both live or die by their wording**

  The model only knows a tool or a memory through the text you write about it. Design them like prompts, not like plumbing.

THREAD

Same lesson as S5: the win is deciding what enters the window, and how it's phrased — now for actions and recall, not just documents.

TOOLS

A tool schema is a prompt

A tool is only as good as its description — because that description is all the model ever sees of it.

- **Name, description, typed parameters**

That's all the model sees. From this text alone it decides whether to call the tool and how to fill the arguments.

- **Vague description → wrong call**

'Gets data' tells the model nothing. Wrong tool, wrong arguments, silent failures follow.

- **Say when to use it — and when not**

The description is where you scope the tool. Include an example argument to anchor the format.

tool definition

```
{
  "name": "get_weather",
  "description":
    "Current weather for a city",
  "parameters": {
     city: string (required) }
}
```

MECHANICS

The tool-calling loop

Here's the mechanic that trips people up: the model asks, but your code is what actually acts.

1

Model proposes

Instead of answering, the model returns a structured request: which tool, with which arguments.

2

You execute

Your code validates the arguments, runs the actual function, and captures the result. The model never runs anything itself.

3

Return the result

Feed the function's output back into the context as a new message the model can read.

4

Continue

The model uses the result to answer, or to decide on another tool call. Loop until done.

KEY INSIGHT

Tool descriptions ARE prompts

If you remember one thing about agents: when it picks the wrong tool, fix the words, not the wiring.

- **The description is the whole interface**

The model's only knowledge of a tool is its name and description. Debugging tool use means editing that text.

- **Give hints and one example**

Argument descriptions and a sample call sharply improve how reliably the model uses the tool.

- **Measure tool-selection accuracy**

Treat 'did it pick the right tool?' as an eval metric, scored on a set — same discipline as S3.

RULE

If the agent picks the wrong tool, fix the words, not the wiring. The description is the bug more often than the code is.

MEMORY

Memory as curated context

Memory isn't a database you dump into — it's a careful choice about what's worth carrying forward.

Semantic

Durable facts about the user or world: 'prefers metric units', 'works in finance'. Re-injected when relevant.

Episodic

Records of past interactions — what happened and when — that inform future turns.

Procedural

How-to routines and instructions the agent should follow consistently across sessions.

Curate ruthlessly

Write only what's durable; retrieve only what's relevant. Stale or bloated memory hurts more than no memory.

AGENTS

Prompting inside an agent

An agent is really just a loop steered by prompts — good news, because prompts are what you already know how to fix.

- **The system prompt sets the mission**

Goal, constraints and boundaries live here. A loose system prompt gives a wandering agent.

- **Tool descriptions steer the path**

Which tool the agent reaches for is decided by their wording — so the descriptions shape the whole trajectory.

- **State clutter is context clutter**

Everything accumulated in the loop competes for attention. Keep the working context lean or quality decays.

TAKEAWAY

Most agent misbehaviour is a prompt or description problem, not a code problem. Reach for the text before the debugger.

HANDS-ON LAB

Lab — Tools & their descriptions

HOSTED · OPENAI

`prompt-engineering-course/06_tools_memory`

TASKS

1. Define a tool with a typed schema and a clear description
2. Wire the model to call it and handle the returned result
3. Break the description on purpose; watch tool selection fail
4. Measure tool-selection accuracy before and after the fix

DELIVERABLE

A working tool call plus a measured example of how a better description improved tool-selection accuracy.

ASSESSMENT

Capstone brief

Time to point all of this at something real — here's what you'll build and present.

1

Build

A small prompt-driven app: a grounded RAG bot, a structured extractor, or a simple tool-using agent.

2

Document your prompts

Show the prompts and, crucially, the iterations that moved the numbers — your S3 discipline applied.

3

Evaluate

An eval set, an LLM-as-judge check, and at least one prompt-injection test case (S8).

4

Deliver

A team repo, work integrated via reviewed pull requests, submitted as a tagged release. Present in Session 9.

WRAP-UP

Recap — Session 6

- ✓ Tools and memory are context you design
- ✓ A tool description is a prompt — measure its effect
- ✓ Memory is curated context: write durable, retrieve relevant
- ✓ Steer agents through prompts, not plumbing

NEXT SESSION

Cost, latency & model choice

Prompts have a bill. Token economics, caching, and picking the right-sized model for the job.